Hadoop ecosystem

Vũ Tuyết Trinh
SoICT - HUST

1

We need a system that scales

- Traditional tools are overwhelmed
  - Slow disks, unreliable machines, parallelism is not easy
- 3 challenges
  - **Reliable storage**
  - **Powerful data processing**
  - Efficient visualization



2

2

What is Apache Hadoop?

- **Scalable** and **economical** data **storage** and **processing**
  - The Apache Hadoop software library is a framework that allows for the **distributed processing** of large data sets across clusters of computers using **simple programming models**. It is designed to **scale out** from single servers to thousands of machines, each offering local computation and storage. Rather than rely on hardware to deliver high-availability, the library itself is designed to detect and **handle failures** at the application layer, so delivering a highly-available service on top of a cluster of computers, each of which may be prone to failures **(commodity hardware)**.
- Heavily inspired by Google data architecture

3

3

Hadoop main components

- Storage: Hadoop distributed file system (HDFS)
- Processing: MapReduce framework
- System utilities:
  - Hadoop Common: The common utilities that support the other Hadoop modules.
  - Hadoop YARN: A framework for job scheduling and cluster resource management.

4

4

Scalability

- Distributed by design
  - Hadoop can run on cluster
- Individual servers within a cluster are called nodes
  - each node may both store and process data
- Scale out by adding more nodes to increase scalability
  - Up to several thousand nodes

5

5

Fault tolerance

- Cluster of commodity servers
  - Hardware failure is the norm rather than the exception
  - Built with redundancy
- File loaded into HDFS are replicated across nodes in the cluster
  - If a node failed, its data is re-replicated using one of the copies
- Data processing jobs are broken into individual tasks
  - Each task takes a small amount of data as input
  - Parallel tasks execution
  - Failed tasks also get rescheduled elsewhere
- Routine failures are handled automatically without any loss of data

6

6

Hadoop distributed file system

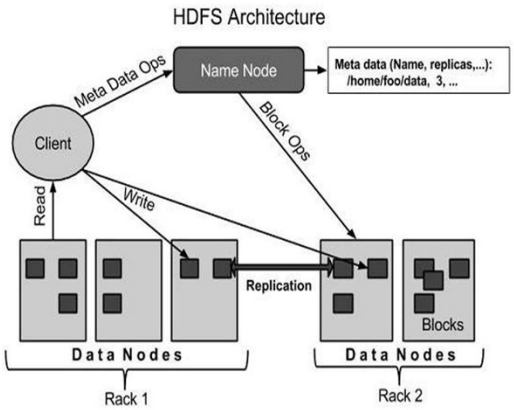
- Provides inexpensive and reliable storage for massive amounts of data
- Optimized for big files (100 MB to several TBs file sizes)
- Hierarchical UNIX style file system
  - (e.g., /hust/soict/hello.txt)
  - UNIX style file ownership and permissions
- There are also some major deviations from UNIX
  - Append only
  - Write once read many times

7

7

HDFS Architecture

- Master/slave architecture
- HDFS master: namenode
  - Manage namespace and metadata
  - Monitor datanode
- HDFS slave: datanode
  - Handle read/write the actual data



8

8

HDFS main design principles

- I/O pattern
  - Append only → reduce synchronization
- Data distribution
  - File is splitted in big chunks (64 MB)

  → reduce metadata size

  → reduce network communication
- Data replication
  - Each chunk is usually replicated in 3 different nodes
- Fault tolerance
  - Data node: re-replication
  - Name node
    - Secondary namenode
    - Enqury data nodes instead of complex checkpointing scheme

9

Data processing: MapReduce

- MapReduce framework is the Hadoop default data processing engine
- MapReduce is a programming model for data processing
  - it is not a language, a style of processing data created by Google
- The beauty of MapReduce
  - Simplicity
  - Flexibility
  - Scalability

10

10

a MR job = {Isolated Tasks}n

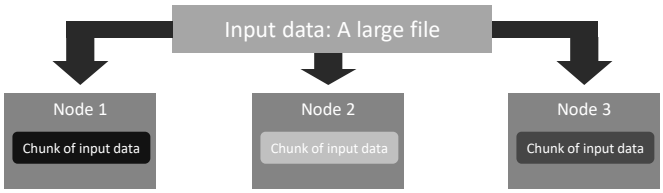
- MapReduce divides the workload into multiple *independent tasks* and schedule them across cluster nodes
- A work performed by each task is done *in isolation* from one another for scalability reasons
  - The communication overhead required to keep the data on the nodes synchronized at all times would prevent the model from performing reliably and efficiently at large scale

11

11

Data Distribution

- In a MapReduce cluster, data is usually managed by a distributed file systems (e.g., HDFS)
- Move code to data and not data to code

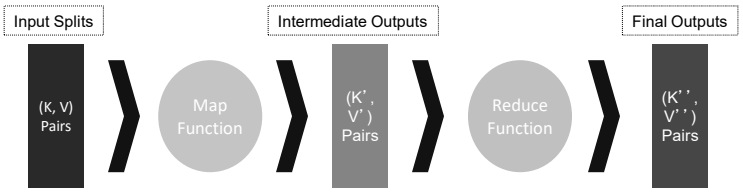


12

12

Keys and Values

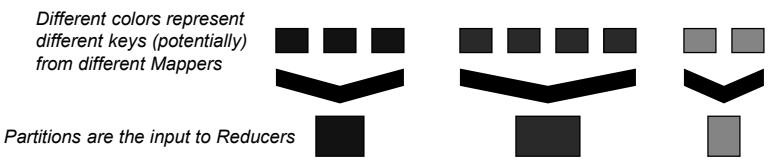
- The programmer in MapReduce has to specify two functions, the *map function* and the *reduce function* that implement the Mapper and the Reducer in a MapReduce program
- In MapReduce data elements are always structured as key-value (i.e., (K, V)) pairs
- The map and reduce functions receive and *emit* (K, V) pairs



13

Partitions

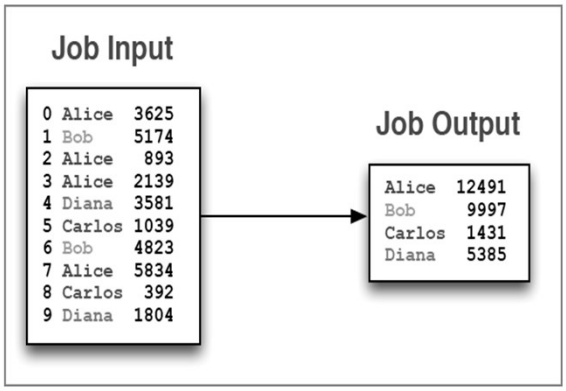
- A different subset of intermediate key space is assigned to each Reducer
- These subsets are known as *partitions*



14

MapReduce example

- Input: text file containing order ID, employee name, and sale amount
- Output: sum of all sales per employee

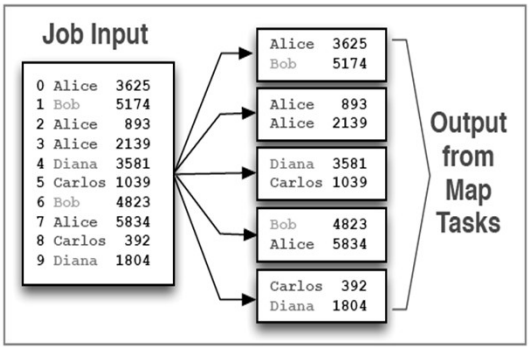


15

15

Map phase

- Hadoop splits job into many individual map tasks
  - Number of map tasks is determined by the amount of input data
  - Each map task receives a portion of the overall job input to process
  - Mappers process one input record at a time
  - For each input record, they emit zero or more records as output
- In this case, the map task simply parses the input record
  - And then emits the name and price fields for each as output



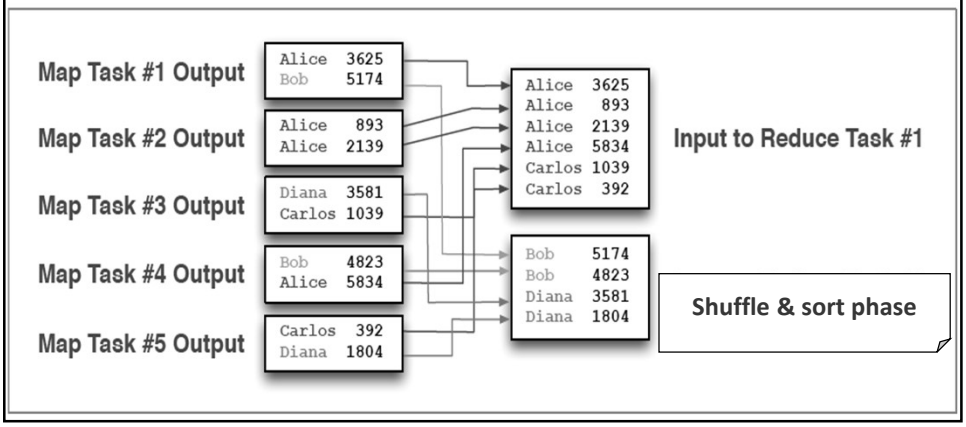
Map phase

16

16

Shuffle & sort

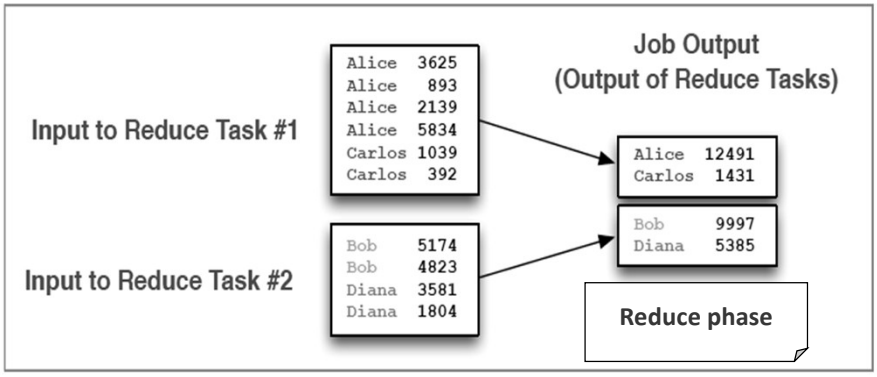
- Hadoop automatically sorts and merges output from all map tasks
  - This intermediate process is known as the **shuffle and sort**
  - The result is supplied to reduce tasks



17

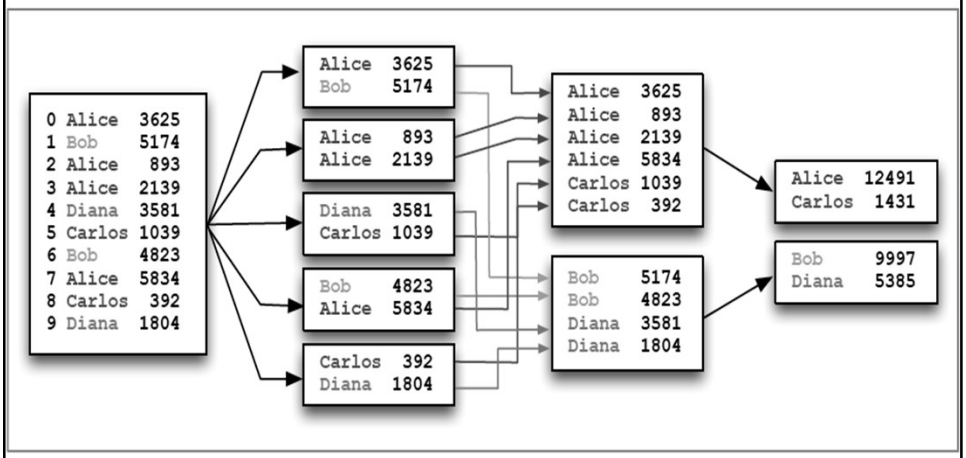
Reduce phase

- Reducer input comes from the shuffle and sort process
  - As with map, the reduce function receives one record at a time
  - A given reducer receives all records for a given key
  - For each input record, reduce can emit zero or more output records
- Our reduce function simply sums total per person
  - And emits employee name (key) and total (value) as output



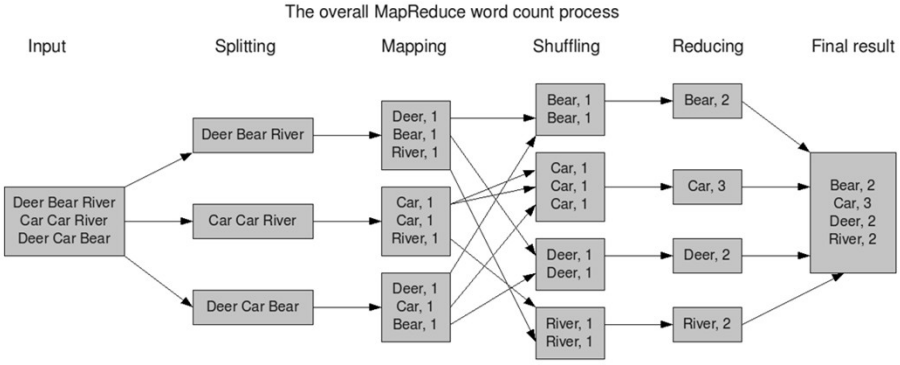
18

Data flow for the entire MapReduce job



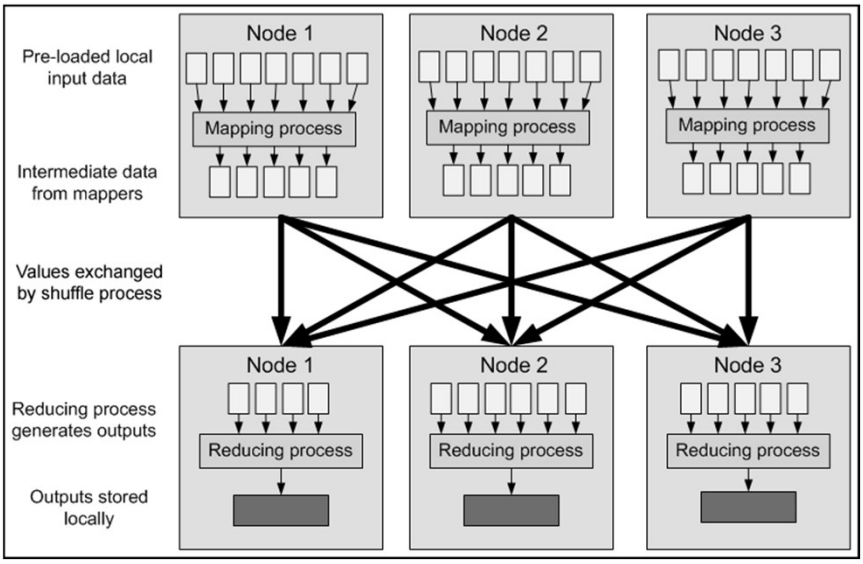
19

Word Count Dataflow



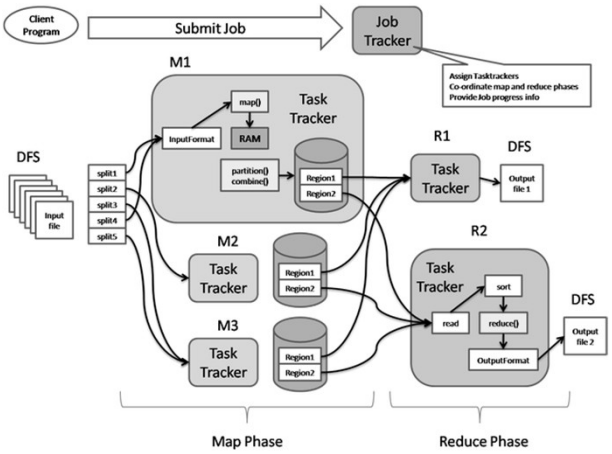
20

MapReduce - Dataflow



21

Map reduce life cycle



22

Example: Word Count (1)

```
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.GenericOptionsParser;



public class WordCount {
public static void main(String [] args) throws Exception
{
Configuration c=new Configuration();
String[] files=new GenericOptionsParser(c,args).getRemainingArgs();
Path input=new Path(files[0]);
Path output=new Path(files[1]);
Job j=new Job(c,"wordcount");
j.setJarByClass(WordCount.class);
j.setMapperClass(MapForWordCount.class);
j.setReducerClass(ReduceForWordCount.class);
j.setOutputKeyClass(Text.class);
j.setOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(j, input);
FileOutputFormat.setOutputPath(j, output);
System.exit(j.waitForCompletion(true)?0:1);
}
```

23

Example: Word Count (2)

```
public static class MapForWordCount extends Mapper<LongWritable, Text, Text, IntWritable>{
public void map(LongWritable key, Text value, Context con) throws IOException, InterruptedException
{
String line = value.toString();
String[] words=line.split(",");
for(String word: words )
{
      Text outputKey = new Text(word.toUpperCase().trim());
  IntWritable outputValue = new IntWritable(1);
  con.write(outputKey, outputValue);
}
}
}

public static class ReduceForWordCount extends Reducer<Text, IntWritable, Text, IntWritable>
{
public void reduce(Text word, Iterable<IntWritable> values, Context con) throws IOException, InterruptedE
{
int sum = 0;
   for(IntWritable value : values)
   {
   sum += value.get();
   }
   con.write(word, new IntWritable(sum));
}
```

24

Hadoop ecosystem

- Many related tools integrate with Hadoop
  - Data analysis
  - Database integration
  - Workflow management
- These are not considered 'core Hadoop'
  - Rather, they are part of the 'Hadoop ecosystem'
  - Many are also open source Apache projects

25

25

Apache Pig

- Apache Pig builds on Hadoop to offer high level data processin
  - Pig is especially good at joining and transforming data
- The Pig interpreter runs on the client machine
  - Turns PigLatin scripts into MapReduce jobs
  - Submits those jobs to the cluster



```
people = LOAD '/user/training/customers' AS (cust_id, name);
orders = LOAD '/user/training/orders' AS (ord_id, cust_id, cost);
groups = GROUP orders BY cust_id;
totals = FOREACH groups GENERATE group, SUM(orders.cost) AS t;
result = JOIN totals BY group, people BY cust_id;
DUMP result;
```

26

26

Apache Hive

- Another abstraction on top of MapReduce
  - Reduce development time
  - HiveQL: SQL-like language
- The Hive interpreter runs on the client machine
  - Turns HiveQL scripts into MapReduce jobs
  - Submits those jobs to the cluster




```
SELECT customers.cust_id, SUM(cost) AS total
  FROM customers
  JOIN orders
    ON customers.cust_id = orders.cust_id
GROUP BY customers.cust_id
ORDER BY total DESC;
```

27

27

Apache Hbase

- HBase is a distributed column-oriented data store built on top of HDFS
  - Is considered as the Hadoop database
- Data is logically organized into tables, rows and columns
  - terabytes, and even petabytes of data in a table
  - Tables can have many thousands of columns
- Scales to provide very high write throughput
  - Hundreds of thousands of inserts per second
- Fairly primitive when compared to RDBMS
  - NoSQL : There is no high/level query language
  - Use API to scan / get / put values based on keys

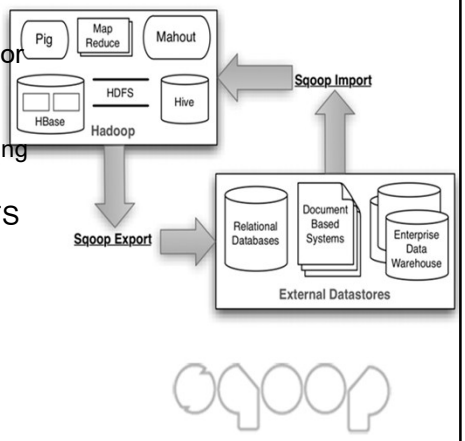


28

28

Apache sqoop

- Sqoop is a tool designed for efficiently transferring bulk data between Apache Hadoop and structured datastores such as relational databases.
- It can import all tables, a single table, or a portion of a table into HDFS
  - Via a Map/only MapReduce job
  - Result is a directory in HDFS containing comma/delimited text files
- Sqoop can also export data from HDFS back to the database



29

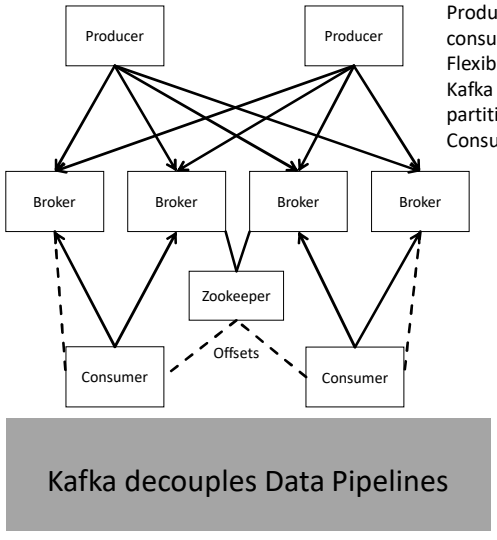
29

Apache Kafka

Producers

Kafka Cluster

Consumers



Kafka decouple data streams
Producers don't know about consumers
Flexible message consumption
Kafka broker delegates log partition offset (location) to Consumers (clients)

Kafka decouples Data Pipelines

trungtv@soict.hust.edu.vn

30

Apache Oozie

- Oozie is a workflow scheduler system to manage Apache Hadoop jobs.
- Oozie Workflow jobs are Directed Acyclical Graphs (DAGs) of actions.
- Oozie supports many workflow actions, including
  - Executing MapReduce jobs
  - Running Pig or Hive scripts
  - Executing standard Java or shell programs
  - Manipulating data via HDFS commands
  - Running remote commands with SSH
  - Sending e/mail messages



31

31

Apache Zookeeper

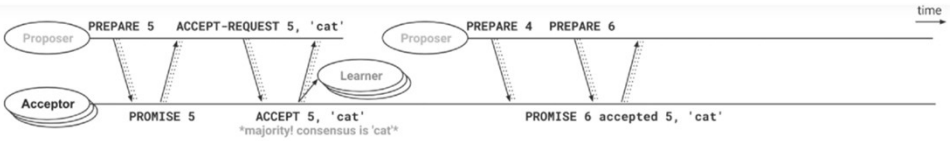
- Apache ZooKeeper is a highly reliable distributed coordination service
  - Group membership
  - Leader election
  - Dynamic Configuration
  - Status monitoring
- All of these kinds of services are used in some form or another by distributed applications



32

32

PAXOS algorithm



⇨ Proposer wants to propose a certain value:
It sends **PREPARE** IDp to a majority (or all) of **Acceptors**.
IDp must be unique, e.g. slotted timestamp in nanoseconds.
e.g. Proposer 1 chooses IDs 1, 3, 5...
Proposer 2 chooses IDs 2, 4, 6..., etc.
Timeout? retry with a new (higher) IDp.

⇨ **Acceptor** receives a **PREPARE** message for IDp:
Did it promise to ignore requests with this IDp?
Yes -> then ignore
No -> Will promise to ignore any request lower than IDp.
Has it ever accepted anything? (assume accepted ID=**IDa**)
Yes ->Reply with **PROMISE IDp accepted IDa, value**.
No -> Reply with **PROMISE IDp**.

1 **If a majority of acceptors promise, no ID<IDp can make it through.**

⇨ Proposer gets majority of **PROMISE** messages for a specific IDp:
It sends **ACCEPT-REQUEST IDp, VALUE** to a majority (or all) of **Acceptors**.
Has it got any already accepted value from promises?
Yes - > It picks the value with the highest **IDa** that it got.
No -> It picks any value it wants.

⇨ **Acceptor** receives an **ACCEPT-REQUEST** message for IDp, value:
Did it promise to ignore requests with this IDp?
Yes -> then ignore
No -> Reply with **ACCEPT IDp, value**. Also send it to all Learners.

2 **If a majority of acceptors accept IDp, value, consensus is reached.**
Consensus is and will always be on value (not necessarily IDp).

⇨ Proposer or Learner get **ACCEPT** messages for IDp, value:

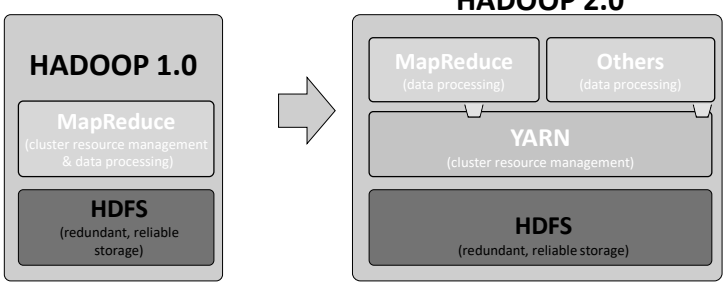
3 **If a proposer/learner gets majority of accept for a specific IDp,**
they know that consensus has been reached on value (not IDp).

https://www.youtube.com/watch?v=d7nAGI_NZPk

33

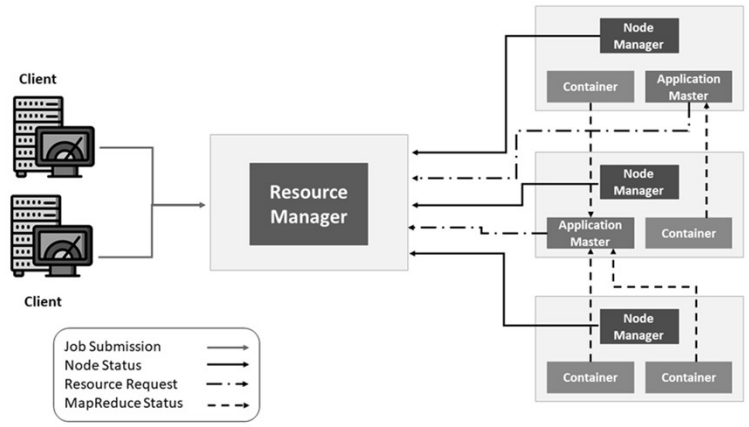
YARN – Yet Another Resource Negotiator

- Nodes have "resources" – memory and CPU cores – which are allocated to application when requested
- Moving beyond Map Reduce
  - MR and non-MR running on the same cluster
  - Most jobtracker functions moved to application masters



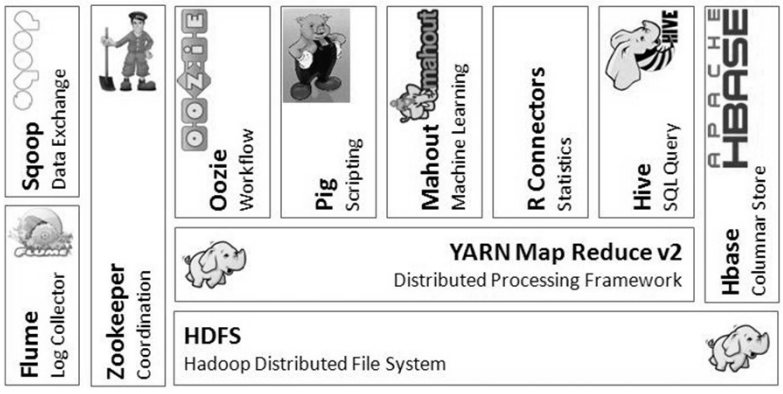
34

YARN execution



35

Big data platform: Hadoop ecosystem



36

Big data management



37

Reference

- White, Tom. *Hadoop: The definitive guide*. " O'Reilly Media, Inc.", 2012.
- Borthakur, Dhruba. "HDFS architecture guide." *Hadoop Apache Project* 53.1-13 (2008): 2.
- Ghemawat, Sanjay, Howard Gobioff, and Shun-Tak Leung. "The Google file system." *Proceedings of the nineteenth ACM symposium on Operating systems principles*. 2003.
- Hunt, Patrick, et al. "ZooKeeper: Wait-free Coordination for Internet-scale Systems." *USENIX annual technical conference*. Vol. 8. No. 9. 2010.
- Dean, Jeffrey, and Sanjay Ghemawat. "MapReduce: simplified data processing on large clusters." *Communications of the ACM* 51.1 (2008): 107-113.

38